

Pair C - Vehicles

Adapted from Yakov's Codebase Reading and Defence Guide (Sept 2026), split per pair by Henry. Full PDF: [docs/COMP2000_Traffic_Simulation_Codebase_Guide.pdf](#).

You are: Ben, Jacob. **You own:** the sealed vehicle hierarchy - all traffic actors and their movement differences.

Files you own

```
src/trafficsim/model/vehicle/  
Vehicle.java          <- sealed abstract base; permits the four below  
Car.java              <- standard movement  
Truck.java            <- heavier, slower acceleration  
Bus.java              <- stops at assigned bus stops  
EmergencyVehicle.java <- siren, ignores red lights  
DriverProfile.java    <- accel multiplier + random slowdown prob  
VehicleVisitor.java   <- generic visitor over the sealed hierarchy
```

Vehicle - the sealed base

`Vehicle` owns the shared state: `x`, `y`, `speed`, `maxSpeed`, `physical length`, `fixed Direction`, `accelStep`, `DriverProfile`, `current Lane`. Only `Car`, `Truck`, `Bus`, `EmergencyVehicle` may extend it.

Class	Max speed	Length	Special behaviour
Car	3.0	18	Standard movement.
Truck	2.0	28	Cargo weight reduces acceleration.
Bus	2.5	32	Stops once at an assigned stop, picks up passengers.
EmergencyVehicle	4.5	22	Siren-on ignores red lights but not physical conflicts.

Vehicle.move(SensorReading)

The heart of your code. Called every tick after the sensor snapshot is built.

```
double obstacle      = r.effectiveStopDistance() - 1.0;
double target        = Math.min(maxSpeed, Math.max(0, obstacle));
double effectiveAccel = accelStep * profile.accelMultiplier();

if (target < speed) speed = target;
else if (speed < target) speed = Math.min(target, speed + effectiveAccel);

x += direction.dx() * speed;
y += direction.dy() * speed;
```

Expression	Meaning
<code>effectiveStopDistance - 1.0</code>	1-unit spare buffer. If a hazard is at zero distance, target speed becomes zero.
<code>min(maxSpeed, ...)</code>	A vehicle never exceeds its class maximum, even on an empty road.
<code>max(0, obstacle)</code>	Prevents negative target speed when overlap risk is detected.
<code>accelStep * profile multiplier</code>	Aggressive drivers accelerate more; cautious less.
<code>direction vector * speed</code>	Straight-line movement only. Direction is <code>final</code> ; vehicles never turn.

Braking is immediate; acceleration is gradual. When `target < speed` , `speed` snaps to `target` on the same tick. Conservative choice - prevents overlap when a hazard suddenly appears. Acceleration adds `effectiveAccel` per tick, capped at `target` .

Driver profiles

Profile	Accel multiplier	Slowdown probability	Effect
AGGRESSIVE	1.6	0.008	Starts faster, rarely slows randomly.
NORMAL	1.0	0.020	Baseline.
CAUTIOUS	0.7	0.040	Starts gently, slows more often.

Random slowdown is deliberately small - creates imperfect driver behaviour and uneven queues without replacing the safety logic. **Emergency vehicles override it with probability zero.**

Bus behaviour

When a bus is spawned, it is given every `BusStop` where `serves(1ane)` is true.

- If it has an assigned stop, it stops within an 8-unit radius, picks up up to 3 passengers, dwells for `DWELL_TICKS` (30 ticks).
- `servedStops` prevents repeated stopping at the same stop.
- If a bus has **no** matching stop for its spawn lane, it continues straight through the network. This is intentional - the spawner picks lanes randomly and we preserve full spawn variety rather than restricting buses to lanes that happen to have stops.

Emergency vehicle

`EmergencyVehicle.stopsAtRedLight()` returns `!sirenOn`. With siren on, the red-light distance is ignored, **but same-lane distance and occupied-intersection distance are still checked**. This is the exact meaning of emergency priority without collision immunity.

Where you touch other pairs

- **Pair A (Engine):** Their `Vehicle.move(reading)` call is your entry point every tick. They pass you a `SensorReading`; you never see the other vehicles directly.
- **Pair B (Network):** You're contained in a `Lane`. You hold a back-ref via `attachTo(lane)`. To leave a lane, someone calls `lane.removeVehicle(this)` - you don't self-remove.
- **Pair D (Lights):** You don't call lights directly - `TrafficSensor` reads the light on your behalf and gives you a `distToRedStop` in the `SensorReading`.

Your defence questions

Do emergency vehicles ignore all safety rules? No. A siren-on emergency vehicle ignores red lights only. It still respects same-lane spacing and waits for occupied or conflicting intersections.

Why do vehicles never turn? The simplified requirement is straight-through traffic. `Direction` is `final` in `Vehicle`, and `move` only adds `direction * speed`. There is no lane-transfer or direction-change code.

Why is braking immediate but acceleration gradual? Safety. If a hazard appears (car ahead brakes, light turns yellow) we must slow *this* tick or we'd overlap. Acceleration doesn't need to be instant - realism benefits from gradual pickup.

Why do buses sometimes not stop? Buses may spawn on any valid boundary lane. They only receive a stop if their exact lane serves a configured stop; otherwise they continue straight. This preserves spawn randomness without constraining bus routes to a subset of lanes.

Why sealed classes for `Vehicle`? Compiler-enforced exhaustiveness. The engine can call `move(reading)` polymorphically; the visitor can pattern-match on the four subclasses without a default branch and without fear a fifth subclass gets added elsewhere.

What's the point of `DriverProfile`? Makes behaviour variety data-driven instead of subclass-driven. We could have `AggressiveCar` / `CautiousCar` / `NormalCar`, but that's a class explosion. Composing profile into `Vehicle` gives 4 vehicle types × 3 profiles = 12 combos without subclassing.

Patterns you're defending

- **Inheritance + polymorphism** - `Vehicle` -> `Car` / `Truck` / `Bus` / `EmergencyVehicle`
- **Sealed classes** - `Vehicle` closes the hierarchy at compile time
- **Visitor** - `VehicleVisitor<R>` over the sealed types (Pair A uses it for stats)
- **Composition over inheritance** - `DriverProfile` composed rather than subclassed

What to skip in the main PDF

- Section 4 (grid.txt) - Pair B's
- Section 6 (tick loop) - Pair A's
- Section 8.3-8.5 (sensor dot product / intersection checks / emergency sensor) - Pair A's; you get the answer via `SensorReading`
- Section 9 (light state machine) - Pair D's
- Section 11 (rendering) - Pair A's view code